



d8 Architecture

Anexa Bijoy
Samuel Knight
Wasif Mustafa
Leo Durnion
Hamza Ahmad
Emma Fuller

Introduction

Our game “All-Nigher” is a 2D Maze game set in a university. To prioritise our developer resources, we’ll be using the **LibGDX** library, saving us time by not needing to re-invent the wheel. Our architecture will be main **event-driven**, choosing to go this route due to LibGDX’s lack of a main loop^[1].

Design Languages and Tools

To create our class diagrams, we used the Unified Modeling Language (UML), using the PlantUML language to create these diagrams using text. By using a programming language and text to generate these diagrams instead of visual tools and editors, we can create class diagrams much more efficiently. To render the diagrams we used PlantText, an online PlantUML text editor. Alternatively, we could’ve used an offline application to render these diagrams as a safety measure in case we did not have internet access. We chose not to do that because we figured that the ease of an online tool outweighed the probability of losing internet access.

Initial Architecture

Screens

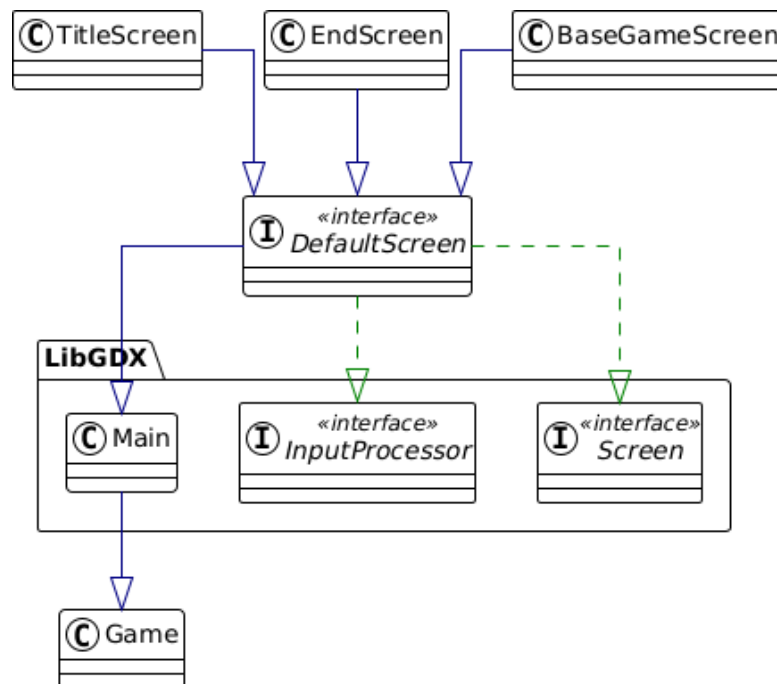


Figure 1: Simple UML class diagram of our game's architecture

We chose to start off with an abstract representation of our game, which we would be able to expand as our needs during development evolved. At the core of our game lies an interface containing key methods and data used across our game's screens. Inside this interface, we placed many of LibGDX's provided classes, such as input processing to detect key presses and mouse clicks, and the standard Screen class. Meanwhile, the Main class, a direct child of LibGDX's Game class, is the heart and entry point of our game. The composite relationship between the Main class (entry point of the game) and the DefaultScreen interface shows how interlinked these two components are, i.e. without the interface containing basic functions like input processing, the game wouldn't be able to exist.

Flow

Figure 2 displays the transitions between each of our game screens. Our game has an **event-driven** architecture, so each of these screens effectively acts as a separate state for our game, making a state diagram a good fit. Further verifying the event-driven nature of our game, LibGDX itself is also event-driven i.e. there is no "main loop" that renders each frame, instead rendering every time an event occurs^[1].

All-Nighter Screen State Transitions

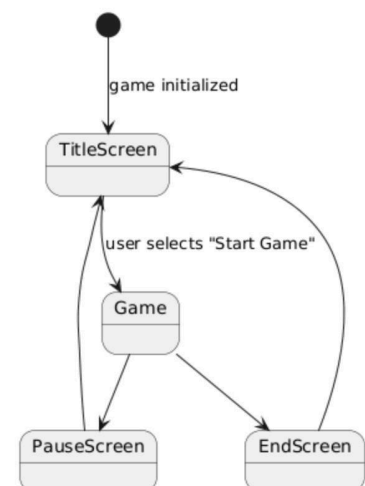


Figure 2: UML state diagram

Final Architecture - Assessment 1

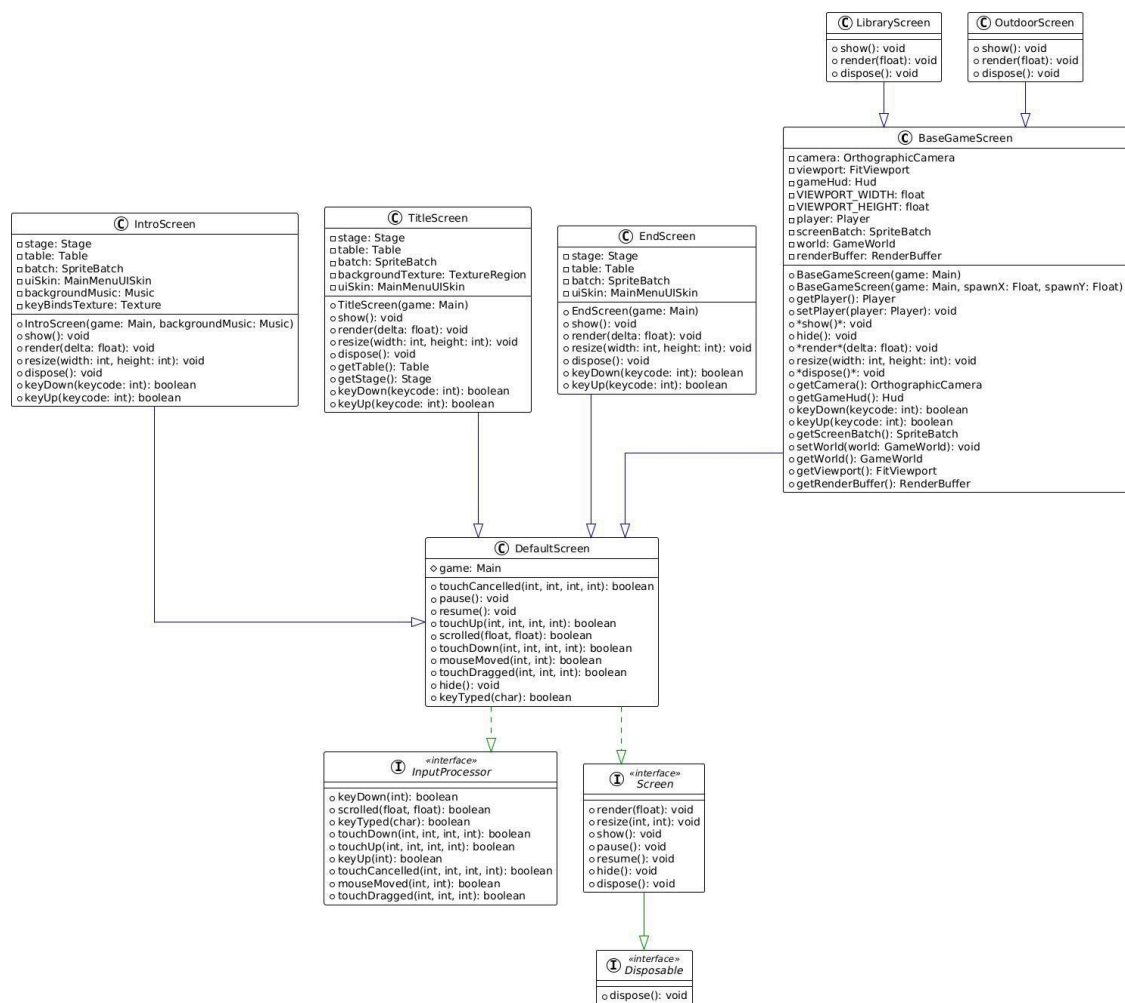


Figure 3: Final class UML diagram of our game screens for Assessment 1

We implemented new screens such as BaseGameScreen which is now a central class of shared attributes (camera, player, world, viewport, HUD, batch) as well as a common setup logic for gameplay. This extends LibraryScreen and OutdoorScreen and they inherit its reusable methods and shared functionality. We also removed PauseScreen as it was a redundant class in implementation due to pause logic being handled by UI overlays of existing screens such as LibraryScreen and OutdoorScreen. Now that pausing the game is built directly into gameplay screens, the system is less complicated, and the player benefits from immediate and seamless transitions.

Moreover, we added the Disposable interface necessitating every screen implements cleanup logic, which avoids memory leaks and ensures proper removal of assets when screen is off or changed, subsequently improving memory efficiency and performance.

Further to the aforementioned, we have expanded the use of interfaces, such as Screen and InputProcessor, so that every screen consistently responds to user actions and manages states effectively. This **event-driven** architecture allows for different screen types to be extended or swapped with ease, while keeping code organised and future testing/improvement simple.

In conclusion, these improvements have made the system more maintainable and modular, as well as making screens more efficient and flexible. The system can be expanded in future and can be relied on for stable and smooth gameplay, as it has adopted a standardised **event-driven** approach, ensured effective resource management, and organised a shared logic across screens^[1].

Updated Screen Architecture - Assessment 2

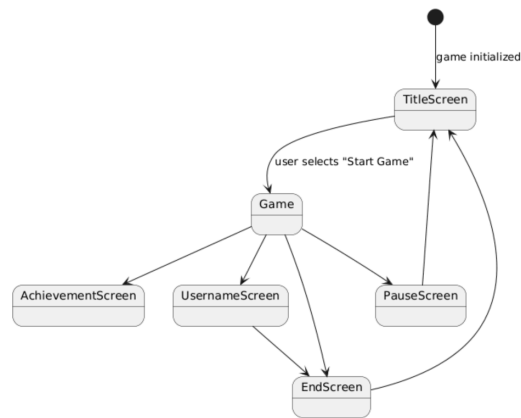


Figure 5: Updated Screen State Transitions

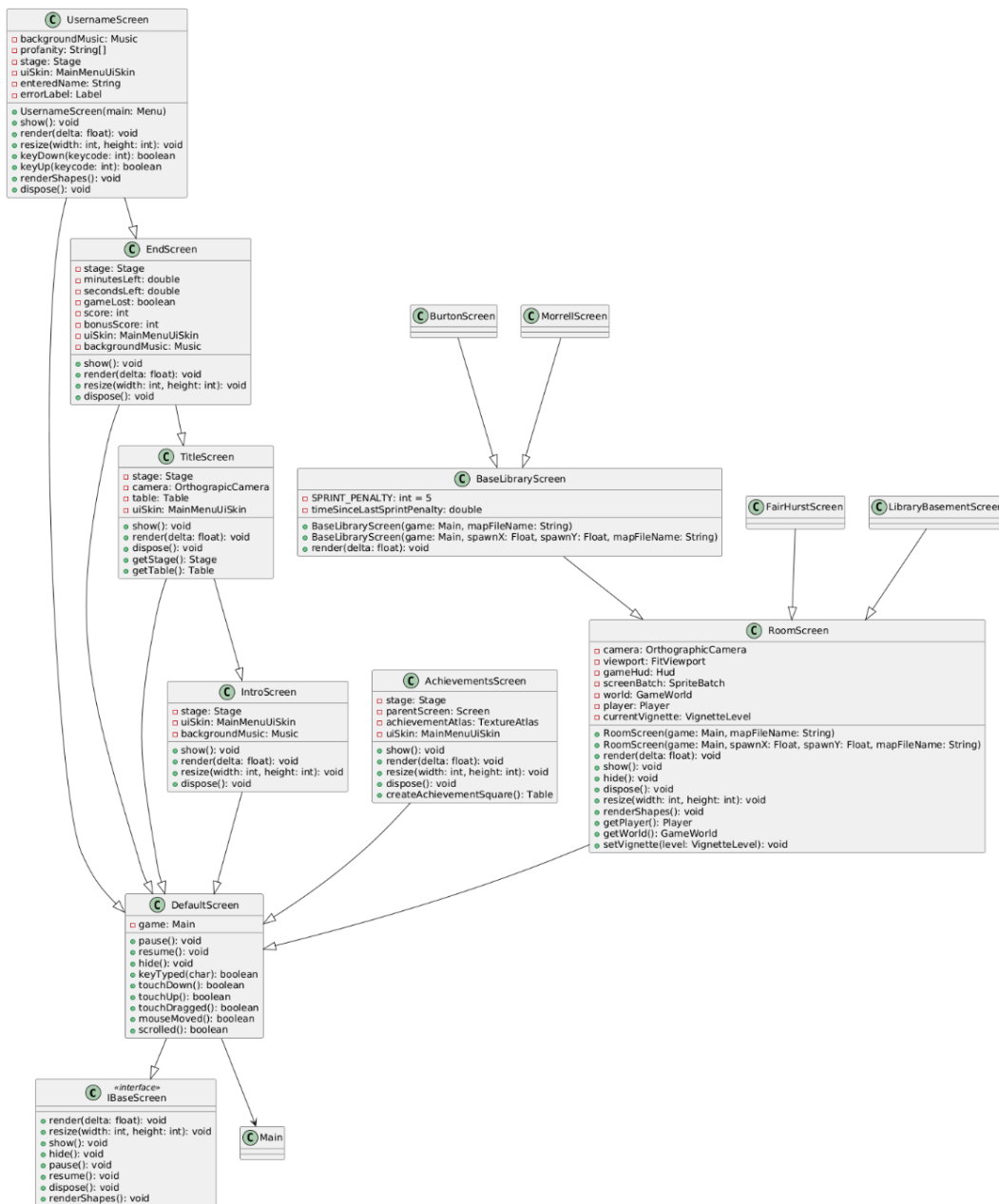


Figure 6: UML class diagram updated with new screen

Expanding upon the previous UML class diagram developed by the previous group, we further extended the screen architecture by implementing new screens like `UsernameScreen`, `AchievementScreen`, `RoomScreen`, and multiple library screens. As the original PlantUML diagram already defined a consistent screen structure, integrating the new rooms was straightforward. Each new room could be added as its own screen whilst following the same rendering and input-handling shown in the original design. Overall, the expanded diagram illustrates how the game moves between different stages of interaction and gameplay through a more detailed system.

Entity Class Architecture - Assessment 1

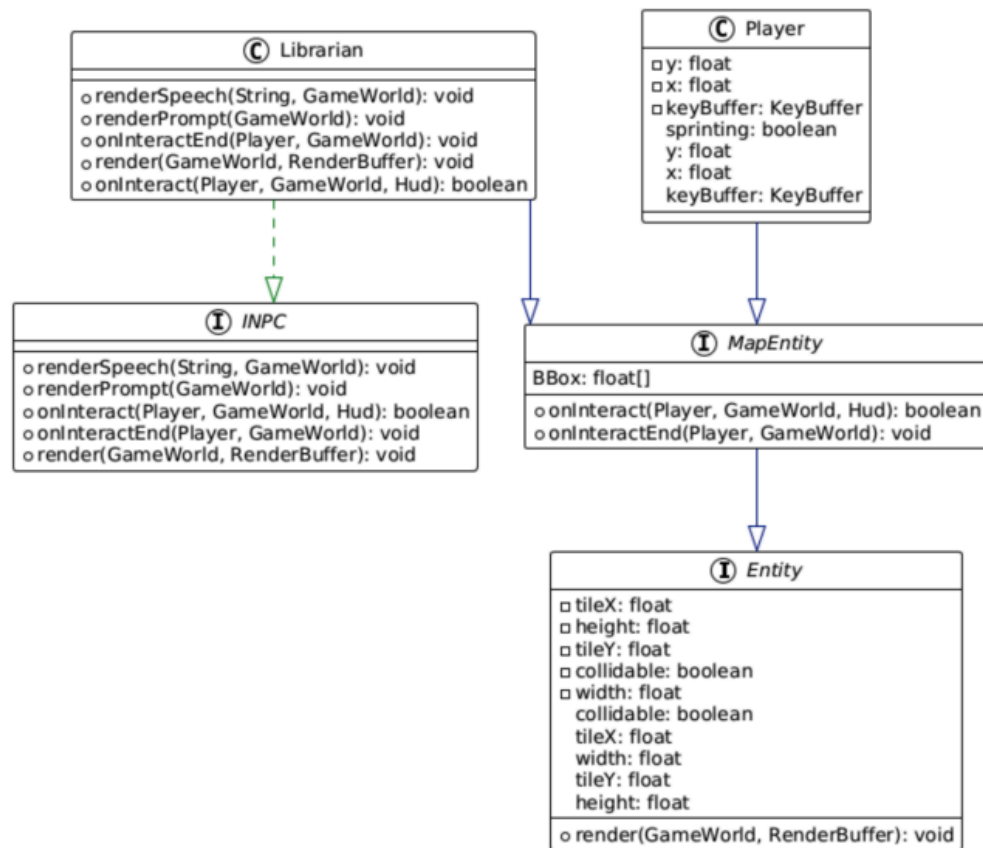


Figure 4: UML class diagram of our entity classes

In addition to our **event-driven** architecture, we implemented some elements of **Entity-Component system (ECS)** architecture.

MapEntity acts as a component for entities that exist inside the world/game map, whilst INPC acts as a separate component to entities which are *not* the player. By using MapEntity and INPC as components, it allows for future expandability to our game through new characters that exist on the map, and crucially, other non-player characters (NPC).

We didn't include ECS in our initial architecture, initially just having a Player class which would react to events. Over time however, it became more visible to us that using an ECS architecture instead of an event-driven architecture would be more viable, once again for its expandability, which would end up proving useful as we added a librarian NPC to our game, which would've been more complicated had we used an independent Player event-driven class^[2].

Entity Class Architecture - Assessment 2

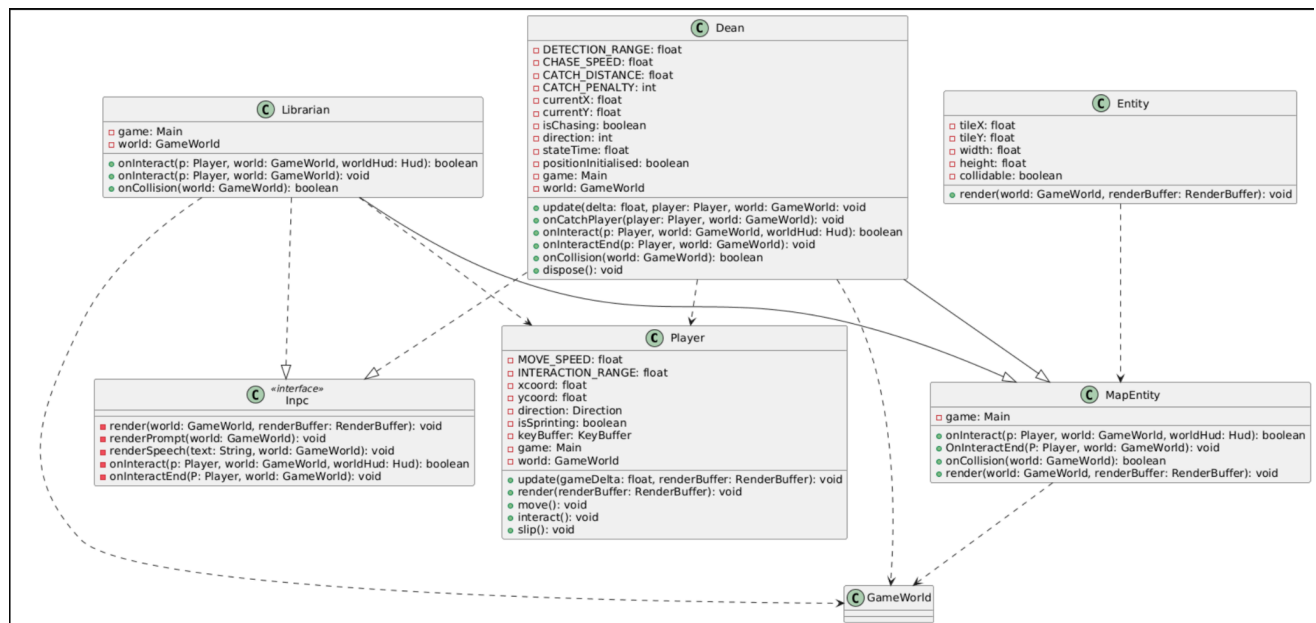


Figure 7: UML class diagram updated with new entities

The updated diagram expands upon the original diagram shown in Figure 4 by adding several new entities and relationships. In addition to the original `Entity`, `MapEntity`, `Player`, `INpc` and `Librarian` relationships, the new diagram introduces `Dean` NPC that also implements the NPC interface, showing support for multiple NPC types with shared behaviour. The diagram further expands class definitions with detailed attributes and Java methods, such as movement, rendering, collision handling, and interaction logic. Overall, the updated diagram enhances the original design by detailing entity behaviour and demonstrating how different NPCs and game objects interact with the system.

Requirements

ID	Architectural Link
UR_SCREEN_SCALABILITY	FitViewport, OrthographicCamera, and resolution-aware screen classes such as BaseGameScreen with its inheriting screens ensure the game scales appropriately and remains legible on different screen sizes.
UR_FAMILY_FRIENDLY	The content itself is inoffensive and neutral by nature, aligning with BBFC PG rating, and is suitable for young children. This is reflected in the architectural implementation across the game classes/interfaces.
UR_UNIVERSITY_ACCURATE/NFR_UNIVERSITY_STUDENT_RELATABLE	There is encapsulation of game state and data of world in GameWorld class, as well as event logic in screen classes, enabling an accurate representation of university environments. Assets are all university themed e.g bookcases, vending machines, etc.. - see Objects in World.
UR_ASSESSMENT1_EVENTS/FR_EVENTS	There is event-driven handling with InputProcessor and the centralised game state/event management inside BaseGameScreen - both supporting event tracking and assessment features - code ensures that at least one type of each event (positive, negative, hidden) occurs.
UR_TIME_TRACKER/FR_TIME_TRACKER	We have the TimerManager class which centralises all timer states, limit enforcements, and HUD time rendering logic for every play session.
UR_MAXIMUM_TIME_LIMIT	TIME_LIMIT attribute set to 300 seconds (5 mins) in TimerManager class.

UR_EVENT_COUNTER/FR_EVENT_COUNTER	Implemented the EventsCounter class to effectively count and track the number of events that occur in a game session, while adjusting HUD accordingly.
UR_COLOURBLIND_FRIENDLY	The Hud class handles UI rendering supporting customisable colouring, ensuring high contrast and accessibility for users with colour blindness.
UR_MULTI_OPERATING_SYSTEM_COMPATIBLE	All classes, game logic, and UI rendering are built using LibGDX, which natively supports MacOS, Windows, and Linux.
UR_BEGINNER_DIFFICULTY	Architecture of the game is designed in such a way where 70% of responses of initial public game testing found the game balanced in regard to difficulty, satisfying the requirement.
UR_LEGIBLE_TEXT	The Hud class displays all on-screen text using scalable, high-contrast fonts using BitmapFont rendering - this ensures legibility across different resolutions and display sizes throughout the game session.
UR_FILE_SIZE	The Disposable interface ensures that resources, such as fonts, textures, and music, are only loaded when needed and are properly removed. This minimises memory leaks and reduces unnecessary memory usage, subsequently keeping the overall file size of the game down.

Bibliography

[1] libGDX, "The life cycle - libGDX," 2025. [Online].

Available: <https://libgdx.com/wiki/app/the-life-cycle>

[2] Wikipedia, "Entity Component System," 2025. [Online].

Available: https://en.wikipedia.org/wiki/Entity_component_system